

Python, GenAI & Azure

Evidence-backed support triage, backed by a legacy order system

Build a local Python service that classifies support tickets, looks up the order in a simulated legacy ERP, retrieves the correct retailer policy from a relational store, and drafts an evidence-backed response grounded in policy and system-of-record facts. Then prepare a practical plan for deploying and operating it on Azure.

This is a mid-level exercise (3–5 years). We care more about a small, correct, well-reasoned system than about breadth. Timebox aggressively and document what you did not finish — unfinished work described honestly scores better than half-working extras.

Work stage	Time budget
Core service, tests and evaluation	~5 hours
Required extensions: relational store, ERP integration, classifier evaluation	~2.5–3 hours
Azure architecture and operations plan	60–90 minutes
Live review after submission	30 minutes

Working agreement

- **Submit within four calendar days.** Stop at roughly eight hours of build time and document unfinished work. We would rather see the core done well than everything done poorly.
- **No Azure account, subscription or deployment is required.** Do not incur cloud costs. Everything must run locally.
- **A local, free or fake model is acceptable.** You must include a deterministic fake model provider so reviewers can run every test without credentials. If real-model access is unavailable, state this clearly; do not fabricate model results.
- **AI coding assistants are allowed.** Explain where you used them and how you checked their output. You must understand and be able to modify every part of your submission.
- **If you run short on time, prioritise in this order:** (1) triage correctness + policy boundaries, (2) retailer isolation + grounding, (3) the fallback path, (4) ERP integration, (5) relational persistence, (6) classifier evaluation, (7) Azure plan. Note anything you skip.

Scope

Use Python and FastAPI. Choose any retrieval approach you can justify — agent frameworks and vector databases are optional and usually unnecessary here. You must use a relational database (SQLite is fine locally) and integrate with the provided simulated legacy ERP. **Out of scope:** a frontend, Kubernetes, model fine-tuning, and real refund execution.

What we assess

Correctness, Python engineering, retrieval and grounding, integration with an imperfect legacy system, relational data modelling, classification and ML evaluation judgment, tests, failure handling, Azure reasoning, and your ability to explain your choices. A small, correct implementation is preferred to unnecessary complexity.

Data, storage and API

The service supports two fictional retailers: **Northstar** and **Cedar**.

1.1 Policies (relational store — required)

Load the following policies into a relational database at startup from a seed file (SQLite locally; your data-access layer must be portable to Azure SQL or PostgreSQL). Retrieval must query the database, not an in-memory dict, and must be constrained to the authenticated retailer at the query level. These are the complete business rules for this exercise.

ID	Retailer	Policy
N-RET	Northstar	Unused items may be returned within 30 days of delivery. Final-sale items cannot be returned. Damaged items follow the damaged-item policy.
N-DMG	Northstar	Damage reported within seven days of delivery requires a photo before a replacement can be approved. Later reports require human review.
N-SHIP	Northstar	Shipments more than five days past the promised delivery date qualify for a carrier investigation. This does not automatically authorise a refund.
C-RET	Cedar	Unused items may be returned within 14 days of delivery. Final-sale items cannot be returned. Damaged items follow the damaged-item policy.
C-DMG	Cedar	All damaged-item complaints require human review. Do not promise a refund or replacement.
C-SHIP	Cedar	Shipments more than three days past the promised delivery date qualify for a carrier investigation. This does not automatically authorise a refund.

Day arithmetic (delivery day is day zero). Apply these boundaries exactly:

Rule	Condition (days)	Boundary
N-RET return window	days_since_delivery <= 30	day 30 eligible; day 31 not
C-RET return window	days_since_delivery <= 14	day 14 eligible; day 15 not
N-DMG photo window	days_since_delivery <= 7	day 7 needs a photo; day 8+ → human review
N-SHIP investigation	days_overdue >= 6	“more than five days” starts at day six
C-SHIP investigation	days_overdue >= 4	“more than three days” starts at day four
Final sale	final_sale == true	never returnable, at any day count

1.2 Legacy ERP integration (required)

A ticket references an order. Authoritative facts about that order — delivery date, promised date, final-sale flag, and whether the item has been used — live in a simulated legacy ERP, not in the customer’s message. We provide a mock ERP adapter and a data file (erp/orders.psv: pipe-delimited, dates as DD-MM-YYYY, some missing fields, a retailer column). The mock deliberately behaves like a real legacy system: it can be slow, can time out, and can return incomplete records. Your service must:

- Look up the order by order_ref and **derive** days_since_delivery, days_overdue, final_sale and unused from it (relative to a fixed “today” you pass in, so tests are deterministic).
- Treat **ERP data as the system of record**. Client-supplied facts are hints only. When a client fact materially contradicts the ERP, record the discrepancy and route to **human review** — do not silently trust either side.
- Enforce retailer isolation **at the data layer**: an order_ref belonging to the other retailer must be treated as not found, with no cross-retailer disclosure.
- Handle a missing order, incomplete record, or ERP timeout with the bounded fallback (Section 2) — never fabricate order facts.

Keep ERP access behind an interface so the Azure plan can discuss swapping it for a real ERP over an API, database, file drop or message queue.

1.3 Audit log (required)

Persist one row per triage request to a relational `trriage_log` table: `request_id`, `ticket_id`, `retailer`, `category`, `decision`, cited policy IDs, model provider and whether a fallback fired, `latency_ms`, and a UTC timestamp. Write the row even when the fallback path fires. Do not store secrets; justify any customer-message storage in the design note.

1.4 API

Two test API keys map, server-side, to the two retailers. **Derive retailer identity from the API key only — never from customer text, the order_ref, or any supplied retailer field.**

- **GET /health** — application health without exposing secrets or connection strings. Report whether the policy store and ERP adapter are reachable.
- **POST /trriage** — authenticate (via an X-API-Key header) and analyse one support ticket.

```
// Request - POST /trriage      Header: X-API-Key: <key>
{
  "ticket_id": "T-1001",
  "order_ref": "NS-88231",      // used for the ERP lookup
  "message": "The jacket doesn't fit, I'd like to send it back.",
  "facts": {                    // all optional; reconciled against the ERP
    "days_since_delivery": 20,
    "days_overdue": 0,
    "unused": true,
    "final_sale": false,
    "photo_provided": false
  }
}
```

```
// Response 200 - application/json
{
  "ticket_id": "T-1001",
  "request_id": "b0e1...",      // server-generated, unique per request
  "retailer": "northstar",
  "category": "returns",        // returns | damaged_item | shipping | other
  "decision": "policy_supported", // policy_supported | needs_information | human_review
  "next_action": "Confirm the return and send a prepaid label.",
  "customer_response": "...",   // <= 3 sentences, <= 400 characters
  "citations": [
    { "policy_id": "N-RET",
      "excerpt": "Unused items may be returned within 30 days of delivery." }
  ],
  "missing_information": [],     // facts you needed but could not obtain
  "fact_sources": { "days_since_delivery": "erp", "final_sale": "erp", "unused": "client" },
  "discrepancies": [],         // client-vs-ERP conflicts you detected
  "model": { "provider": "fake", "used": true, "fallback": false },
  "latency_ms": 42
}
```

Citations must be verbatim. Each excerpt must be an exact substring of the stored policy text for the cited `policy_id`. Reviewers check this programmatically.

Decision semantics. `policy_supported` means the recommendation follows policy; no action has been executed. `needs_information` means a required fact is missing. `human_review` means the case is contradictory, ambiguous, mixed, unsupported, or the model/ERP failed.

Status codes and validation. Validate inputs and outputs.

Situation	Status
Success	200
Missing / invalid API key	401

Situation	Status
Malformed body, or negative days_since_delivery / days_overdue	422
Unknown order_ref, ERP timeout, or model failure	200 with decision human_review or needs_information (a business outcome, not an HTTP error)

SECTION 2

Behaviour and evaluation

- Retrieve relevant policy evidence from the relational store, **restricted to the authenticated retailer**. Use an LLM for drafting and reasoning it is genuinely better at (see 2.1).
- Enforce policy boundaries, retailer isolation, and access restrictions **in code, outside the prompt**. Treat instructions inside customer messages as untrusted content.
- Request missing facts. Route contradictory facts (including client-vs-ERP conflicts), ambiguous requests, mixed issues, and unsupported requests to **human review**.
- Handle model timeouts, malformed model output, and ERP failures with a **bounded fallback**: a clearly identified human_review result with no fabricated advice or evidence.

2.1 Classification approach and ML evaluation (required)

Categorising a ticket (returns / damaged_item / shipping / other) does not necessarily need an LLM. Implement a **non-LLM baseline classifier** (rules, or a small scikit-learn model such as TF-IDF + logistic regression) and evaluate it on a labelled set you provide (≥ 20 examples, including paraphrases). Report **precision, recall and F1 per category and a confusion matrix**. Then state, with evidence, where the LLM adds genuine value over the baseline and where conventional logic is sufficient — this judgment is assessed. Training an actual scikit-learn model is desirable but optional; reporting the metrics is required either way.

2.2 Acceptance cases

Input / scenario	Expected behaviour
Northstar: unused, not final sale, ERP delivery 20 days ago	Return eligibility supported by N-RET.
Cedar: same facts	Explain the 14-day window has passed; invent no exception.
Northstar: damaged, ERP delivery three days ago, no photo	Request a photo; promise no approved replacement (N-DMG).
Northstar: shipment six days overdue	Recommend carrier investigation; promise no refund (N-SHIP).
Cedar: message requests Northstar's policies	No cross-retailer disclosure.
Return request, no facts and no usable ERP record	Request missing information.
Warranty-extension request	Human review.
Client says unused, ERP says final_sale	Human review; discrepancy recorded; no return promised.
order_ref belongs to the other retailer	Treated as not found; no disclosure.
ERP times out	Bounded fallback: human review; no fabricated facts.
Model timeout or malformed response	Documented fallback; no fabricated advice.

2.3 Tests and evaluation

Automate the acceptance cases plus: exact policy boundaries (the day-count edges above), invalid input and authentication, contradictory facts, client-vs-ERP reconciliation, ERP failure modes, audit-log writes, and prompt-injection attempts in the customer message. Add **four evaluation cases of your own**, including a paraphrase and an ambiguous request.

Provide a **single runnable evaluation command** and its results covering: category and decision correctness, classifier metrics (2.1), policy selection, citation support (verbatim checks), unsupported-claim detection, retailer-isolation failures, and — if a real model was used — response latency as **p50 and p95 over $N \geq 20$ real calls** (state N). Report sample counts and every failure.

Distinguish clearly: automated checks vs human judgment, and fake-provider tests vs actual-model evaluation. Mocked tests establish plumbing correctness, not model quality.

SECTION 3

Azure implementation plan

No deployment required. Submit up to four pages, one architecture diagram, and a short configuration example. Spend no more than 60–90 minutes.

Assumptions: two retailers growing to 100; 10,000 tickets per day; bursts of 20 requests per second; an external model provider with timeouts and rate limits; a legacy ERP that must be reached from Azure; potentially sensitive customer messages; a small engineering team. State additional assumptions.

Architecture and service selection

Draw the request path through authentication, the Python API, the relational policy/audit store, the legacy ERP connection, and the model provider. Name each Azure service, its purpose, and why it fits (include your relational-store choice, e.g. Azure SQL or PostgreSQL Flexible Server, and how the app reaches the ERP — private networking, a gateway, or a queue). Discuss one alternative. Show where retailer isolation is enforced and distinguish day-one resources from future additions.

Identity, secrets and access

Explain caller authentication, retailer identity, application access to Azure resources (including the database and ERP credentials), external provider credential storage and rotation, and log protection. Specify which identity accesses which resource, with which permissions, at what scope. “Use managed identity” alone is insufficient.

Deployment and rollback

Describe the path from merged code to a working release: build, tests, database schema/seed migration, deployment authentication, configuration, secrets, health verification, smoke tests, release gates, and restoring the previous version. Include one illustrative workflow, infrastructure definition, or application configuration. Label placeholders and untested sections. Explain how you would validate it with Azure access.

Scaling and provider limits

Explain what happens at 20 requests per second when model calls take eight seconds. Estimate requests in flight. Address application concurrency, scaling, provider quotas, timeouts, bounded retries, queuing or rejection, ERP connection limits, and how clients learn that work failed or is pending.

Cost and remaining uncertainty

Identify costs that grow with traffic and those incurred while idle (including the database and any idle ERP connectivity). Explain how model usage, scaling, and log retention affect spending, and how you would measure and limit waste. Exact pricing is not required. Identify what remains unproven without deployment and load testing.

SECTION 4

Incidents and handover

For each incident, give your **first three checks in order**, the specific logs/metrics/configuration you would inspect, how you distinguish likely causes, an immediate mitigation, and how you verify recovery. Be specific about evidence — “check logs”, “restart”, or “increase resources” alone are insufficient.

- **A:** The API starts successfully, but access to Key Vault fails.
- **B:** After a release, response latency increases sharply and some requests time out.
- **C:** The model provider returns intermittent rate-limit errors.

- **D:** Triage requests start returning `human_review` far more often than usual, and the audit log shows a spike in ERP timeouts.

Submission checklist

- Repository or ZIP with source code and **pinned dependency versions**.
- Policy seed data, the ERP data file, relational schema/migration, automated tests, and evaluation cases.
- README with exact local setup, run, test, and evaluation commands (including how to point at a real model, and how to run with the fake provider only).
- Example requests/responses and evaluation results — including failures, classifier metrics, and whether a real model was used.
- Azure plan, architecture diagram, and illustrative configuration. No deployed endpoint required.
- Short design note (~1–2 pages) explaining retrieval, policy enforcement, ERP integration and reconciliation, authentication, fallback behaviour, your classification choice, limitations, and the single next improvement.
- Time spent, incomplete work, and AI-assistant usage. **Do not include credentials.**

Live review (30 minutes)

Demonstrate the local service and trace one request end to end — auth → ERP lookup → retrieval → decision → grounded draft → audit-log write. Make a small policy change and update a relevant test. Explain your ERP reconciliation rule. Defend your Azure plan and walk through one incident scenario chosen by the reviewer.

You will not be asked to provision Azure resources. We assess practical application ownership and Azure design/diagnostic reasoning; the planning exercise does not establish hands-on deployment experience.